

Комплексный подход к дедуктивной верификации реализаций умножения матриц

По итогам участия в Летнем Системном Буткемпе от лаборатории YADRO, НГУ

Семинар теоретического и экспериментального программирования им. В.А. Непомнящего

22 сентября 2026 г.

На пути к minigemm

Верифицировано ранее:

- Версия 0. Наивная реализация.
- Версия 1. С переставленными внутренними циклами.

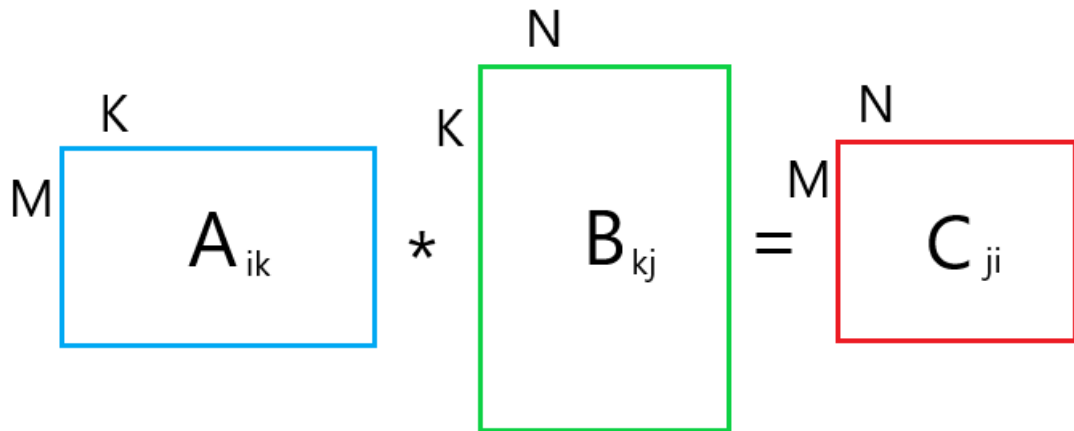
Работа в рамках Летнего Буткемпа лаборатории YADRO:

- Версия 2. С ассемблерными вставками.

В течение этого семестра (до конца 2026 года):

- Версия 3. С микроядром.

Визуализация умножения матриц



Версия 0. Наивная реализация

```
void gemm_v0(int M, int N, int K, const float * A, const float * B, float * C)
{
    for (int i = 0; i < M; ++i)
    {
        for (int j = 0; j < N; ++j)
        {
            C[i*N + j] = 0;
            for (int k = 0; k < K; ++k)
                C[i*N + j] += A[i*K + k] * B[k*N + j];
        }
    }
}
```

Версия 0. Наивная реализация

```
1  void gemm_v0(int M, int N, int K, const float* A, const float* B, float* C)
2  {
3      for (int i = 0; i < M; ++i)
4      {
5          for (int j = 0; j < N; ++j)
6          {
7              C[i*N + j] = 0;
8              for (int k = 0; k < K; ++k)
9                  C[i*N + j] += A[i*K + k] * B[k*N + j];
10         }
11     }
12 }
```

Версия 1. С переставленными внутренними циклами

```
void gemm_v1(int M, int N, int K, const float * A, const float * B, float * C)
{
    for (int i = 0; i < M; ++i)
    {
        float * c = C + i * N;
        for (int j = 0; j < N; ++j)
            c[j] = 0;
        for (int k = 0; k < K; ++k)
        {
            const float * b = B + k * N;
            float a = A[i*K + k];
            for (int j = 0; j < N; ++j)
                c[j] += a * b[j];
        }
    }
}
```

Версия 1. С переставленными внутренними циклами

```
1  void gemm_v1(int M, int N, int K, const float* A, const float* B, float* C)
2  {
3      for (int i = 0; i < M; ++i)
4      {
5          float* c = C + i * N;
6          for (int j = 0; j < N; ++j)
7              c[j] = 0;
8          for (int k = 0; k < K; ++k)
9          {
10             const float* b = B + k * N;
11             float a = A[i*K + k];
12             for (int j = 0; j < N; ++j)
13                 c[j] += a * b[j];
14         }
15     }
16 }
```

Версия 2. С ассемблерными вставками

```
void gemm_v2(int M, int N, int K, const float * A, const float * B, float * C)
{
    for (int i = 0; i < M; ++i)
    {
        float * c = C + i * N;
        for (int j = 0; j < N; j += 8)
            _mm256_storeu_ps(c + j + 0, _mm256_setzero_ps());
        for (int k = 0; k < K; ++k)
        {
            const float * b = B + k * N;
            __m256 a = _mm256_set1_ps(A[i*K + k]);
            for (int j = 0; j < N; j += 16)
            {
                _mm256_storeu_ps(c + j + 0, _mm256_fmadd_ps(a,
                    _mm256_loadu_ps(b + j + 0), _mm256_loadu_ps(c + j + 0)));
                _mm256_storeu_ps(c + j + 8, _mm256_fmadd_ps(a,
                    _mm256_loadu_ps(b + j + 8), _mm256_loadu_ps(c + j + 8)));
            }
        }
    }
}
```


Версия 2. С ассемблерными вставками

```
1 void gemm_v2(int M, int N, int K, const float* A, const float* B, float* C)
2 {
3     for (int i = 0; i < M; ++i)
4     {
5         float* c = C + i * N;
6         for (int j = 0; j < N; j += 8)
7             _mm256_storeu_ps(c + j + 0, _mm256_setzero_ps());
8         for (int k = 0; k < K; ++k)
9         {
10             const float* b = B + k * N;
11             __m256 a = _mm256_set1_ps(A[i*K + k]);
12             for (int j = 0; j < N; j += 16)
13             {
14                 _mm256_storeu_ps(c + j + 0, _mm256_fmadd_ps(a,
15                     _mm256_loadu_ps(b + j + 0), _mm256_loadu_ps(c + j + 0)));
16                 _mm256_storeu_ps(c + j + 8, _mm256_fmadd_ps(a,
17                     _mm256_loadu_ps(b + j + 8), _mm256_loadu_ps(c + j + 8)));
18             }
19         }
20     }
21 }
```

Цели проекта

- Цель 1. Выполнить реинжиниринг процесса дедуктивной верификации реализаций `gemm_v0`, `gemm_v1`

Мотивация

- Повысить уровень автоматизации процесса верификации
- Заложить более надежную теоретическую и инженерную основу для верификации других алгоритмов умножения матриц
- Цель 2. Провести дедуктивную верификацию реализации `gemm_v2`

Мотивация

- Сделать ещё один шаг к верификации реализации алгоритма `minigemm`

- ① С учетом замечаний, сделанных в ходе изучения проделанной ранее работы, были проверены следующие гипотезы:
 - ① ✓ Удалить операцию означивания матрицы C нулями в наивном коде
 - ② ✓ Удалить лишние `separated` в спецификациях
 - ③ ✗ По одному удалять `valid_read` и `valid` в спецификациях
 - ④ ✗ Ослабить посылки в базе индукции индуктивных предикатов на $n < 0$ в теории предметной области
 - ⑤ ✗ По одному удалять `assert` в спецификациях
- ② Подготовлены отчетные материалы

- 1 Изучена предметная область
- 2 Определены спецификации операций, входящих в ассемблерные вставки
- 3 Расширена формальная теория предметной области на функции и предикаты из этих спецификаций
- 4 Порождены условия корректности
- 5 Доказаны условия корректности
- 6 Подготовлены отчетные материалы

- Основы дедуктивной верификации
- Основы машинного доказательства теорем
- Frama-C/WP + Why3
- SMT-решатели(Alt-Ergo, CVC4, CVC5, Z3, Vampire)
- Coq
- LLM для генерации скриптов доказательства
- Эвристики и стратегии доказательства

Определены спецификации операций, входящих в ассемблерные вставки

```
typedef struct { float e[8]; } __mm256;  
/*@  
  assigns \nothing;  
  ensures  $\forall$  integer  $i$ ;  $0 \leq i < 8$   
     $\Rightarrow$   $\text{result.e}[i] == \text{a.e}[i] * \text{b.e}[i] + \text{c.e}[i]$ ;  
*/  
__mm256 __mm256_fmadd_ps(__mm256 a, __mm256 b, __mm256 c);
```

Расширена формальная теория предметной области на функции и предикаты из этих спецификаций

lemma dp_frame{L1,L2}:

```
  ∀ float* A, float* B, integer to, i, K, j, N, real res;  
  (∀ integer idx; 0 ≤ idx < K*(i+1)  
   ⇒ \at(A[idx],L1) == \at(A[idx],L2)) ∧  
  (∀ integer idx; 0 ≤ idx < N*K  
   ⇒ \at(B[idx],L1) == \at(B[idx],L2)) ∧  
  0 ≤ i ∧ 0 ≤ j < N ∧ to ≤ K ⇒  
  \at(DotProduction(A,B,to,i,K,j,N,res),L1)  
  ⇒ \at(DotProduction(A,B,to,i,K,j,N,res),L2);
```

Порождены условия корректности

WP — Goals			
Scope	Property		Status
gemm_v2	Instance of 'Pre-condition' ...		✓ Valid (Qed 181ms) (Alt-Ergo 201ms) (Cached)
gemm_v2	Instance of 'Pre-condition' ...		✓ Valid (Qed 264ms) (Alt-Ergo 316ms) (Cached)
gemm_v2	Instance of 'Pre-condition' ...		✓ Valid (Qed 78ms) (Alt-Ergo 251ms) (Cached)
gemm_v2	Post-condition		✓ Valid (Qed 22ms) (Vampire 95ms) (Cached)
gemm_v2	Exit-condition		✓ Valid (Qed)
gemm_v2	Loop assigns ...		✓ Valid (Qed)
gemm_v2	Loop assigns ...		✓ Valid (Qed)
gemm_v2	Loop assigns ...		✓ Valid (Qed)
gemm_v2	Loop assigns ...		✓ Valid (Qed 135ms) (Alt-Ergo 359ms) (Cached)
gemm_v2	Loop assigns ...		✓ Valid (Qed)
gemm_v2	Loop assigns ...		✓ Valid (Qed)
gemm_v2	Loop assigns ...		✓ Valid (Qed)

Доказаны условия корректности

Решатель	Доказано целей	Время выполнения
Qed (встроенный)	50	1ms – 65ms – 337ms
Alt-Ergo 2.6.3	30	57ms – 493ms – 6.2s
Coq 8.18.0	10	993ms – 1.5s – 3.0s
Vampire 4.5.1	3	47ms – 79ms – 95ms
Z3 4.13.4	2	25ms – 150ms
CVC4 1.8	1	83ms
Итого	96/96	—

- ReadMe с инструкцией по воспроизведению результатов
- Вики с описанием содержимого репозитория
- Файл с формальной спецификацией ассемблерных вставок
- Файл со аннотированной реализацией `gemm_v2`
- Скрипты доказательств условий корректности в формате `Coq`
- Файл со спецификацией предметной области
- Файлы с модифицированными аннотированными реализациями `gemm_v0` и `gemm_v1`



Репозиторий проекта:
<https://github.com/ylab-nsu/sb26-trust-minigemmm/>

- Подготовка статьи по результатам проекта в рейтинговый журнал
- Верификация `gemm_v3`
- ...
- Верификация `minigemm`

Команда проекта



Идея версии 3. С микроядром

